

Building an AI Dental Receptionist

The full rebuild guide, the prompts, and the video script

Fairy Share Dental · ElevenLabs + FastAPI + Open Dental + n8n · September 2026

Everything in this guide was built and run against Open Dental's public developer test database with invented patients. It is a demonstration system and is not HIPAA compliant.

Part 0 — Read this before you film

Your script is strong and the structure works. Three things in it do not match what we actually built. Fix these before recording, because a dentist or a developer in the comments will catch them.

1. n8n does not talk to Open Dental

Your script says: “n8n turns that request into calls Open Dental understands.” In this build it does not. A Python backend (FastAPI) talks to Open Dental. n8n only delivers the confirmation message afterwards (Telegram, email or SMS).

Why it was built that way: booking safely means checking for conflicts, retrying, reading the appointment back and deciding whether it truly exists. That logic belongs in tested code, not in workflow nodes. A half-finished node run could otherwise tell a patient they are booked when they are not.

Say this on camera instead: “The tools call my own backend. That backend is the only thing allowed to touch Open Dental, and it has one rule: check, book, verify, confirm. n8n handles the confirmation message after the booking is verified.” Your line “the model may propose, the workflow decides” still holds — the decider is just the backend.

2. It does not transfer the call to a human yet

Your script says the agent “hands the call to a person”. Human handoff is V2 and is not built. Today it says a team member will call back. Either say that, or build the transfer before filming.

3. The emergency refusal — now tested

As of 2026-09-24 this is verified. Five scripted callers were run against the live agent through ElevenLabs' simulation API, including one who pushed three times for a same-day appointment while describing swelling and trouble breathing. The agent redirected to 911 every time and booked nothing. You can make that claim on camera.

Everything else in your script is true

- The appointment really does land in Open Dental, the same schedule the front desk uses.
- The agent only offers two or three times, never twenty.
- It only says “booked” after the backend reads the appointment back from Open Dental.
- Everything runs on a test database with invented patients.
- The compliance checklist at 10:00 is accurate. Keep it word for word.

Fill in your placeholders

Placeholder	Use this
[AGENT]	Joy
[PRACTICE]	Fairy Share Dental (fictional, Dallas TX)
[PLATFORM]	ElevenLabs Agents
[YOUR REASON]	Open Dental publishes its REST API and hosts a free test database, so I could build against real software without touching a real practice

What is live right now

Everything below is running and was checked on 2026-09-24. None of it depends on your laptop being awake, so you can record whenever you like.

Piece	Where	State
Joy, the agent	elevenlabs.io → Agents → Fairy Share Dental – Receptionist (demo)	Claude Haiku 4.5, 6 tools, knowledge base attached
Backend	https://fairy-share-dental-2.onrender.com	Hosted on Render, deploys from GitHub on push
Proof page	https://fairy-share-dental-2.onrender.com/schedule	Password protected, read-only, phone friendly
Confirmations	n8n workflow → Telegram	Sends only after the appointment is verified
Open Dental	Public developer test database	Live reads and writes; wiped nightly

Before you record: run `backend/scripts/seed_demo_patients.py`. The test database is wiped every night, so without it the schedule is empty and Benjamin Johnson will not be found. Then make one throwaway booking to warm everything up, and delete nothing — a schedule with a couple of appointments on it looks like a real practice.

Part 1 — The system, accurately

```
CALLER
|
v
ElevenLabs agent "Joy" <- voice, listening, talking
| 6 tools (webhooks, HTTPS + shared secret)
v
FastAPI backend (Render) <- THE DECIDER: check, book, verify
| rate limiting, conflict checks, retries
+--> Open Dental Remote API <- source of truth: patients, schedule
|
+--> n8n workflow <- delivers the confirmation message only
|
+--> Telegram / email / SMS
```

The four parts, in the order you will show them

Part	What it does	Where it runs
Joy (the agent)	Listens, speaks, decides which tool to use	ElevenLabs
The 6 tools	The only actions Joy is allowed to take	ElevenLabs → your backend
The backend	Checks the schedule, books, verifies, blocks double bookings	Render (Python)
n8n	Sends the written confirmation after verification	Your n8n instance

The six tools

Tool	What it does
lookup_patient	Finds an existing patient by name + date of birth. Returns only a match status
get_appointments	Lists that patient's upcoming appointments, to avoid duplicates
get_availability	Returns up to 3 real open times, each with a signed, expiring slot id
create_appointment	Re-checks the slot, then books. Can never report "booked"
verify_appointment	Reads it back from Open Dental. This is the only source of "you're booked"
send_confirmation	Re-verifies, then sends the message via n8n

The reason create and verify are separate tools is the whole point of the video. Creating something is not proof it exists. Show that on screen.

Part 2 — Rebuild it, step by step

Thirteen steps. Each has what to do, the exact prompt to give Claude Code, how you know it worked, and what to film. Total working time is roughly a day; on camera it compresses to about 12 minutes.

Step 1 — Research before any code

Do not let the model invent API endpoints. Make it read the real documentation first and report back.

Prompt to paste:

```
I want to build an AI dental receptionist for a fictional practice, Fairy Share Dental in Dallas, Texas. Voice: ElevenLabs. Practice software: Open Dental. Backend: Python + FastAPI.
```

```
Before writing any code, research the CURRENT official Open Dental API documentation and tell me:
```

1. What you found, with links
2. Whether patient lookup, availability, booking and verification are possible on their developer/test environment
3. The exact endpoints we need
4. A simple architecture
5. The ElevenLabs tool definitions
6. A folder structure
7. A step-by-step milestone 1 plan
8. Anything you cannot verify

```
Do not invent endpoints or parameters. If something cannot be verified, say so clearly. Then STOP and wait for my approval.
```

- Done when: you have a written plan naming real endpoints, and a list of unknowns.
- Film: the moment it says an endpoint could not be verified. That is the discipline the video is about.

Step 2 — Check the test database yourself

Open Dental publishes a test key in its API setup documentation. Run the calls yourself before trusting any plan.

```
curl -s -H "Authorization: ODFHIR <devkey>/<customerkey>" \
https://api.opendental.com/api/v1/operatories
```

```
curl -s -H "Authorization: ODFHIR <devkey>/<customerkey>" \
https://api.opendental.com/api/v1/providers
```

- Done when: you can see the rooms and providers in that database.
- Film: this is your “it’s real software, not a mock” proof. Show the raw JSON for two seconds.

In our build, GET /appointments/Slots returned an empty list every time. The test database has no provider schedules, and schedules cannot be created through the API. That discovery shaped the whole availability design — see Step 5 and the “What broke” section.

Step 3 — Put the practice in config, not in code

Hours, rooms, providers and appointment lengths belong in configuration files so a different practice is a config change, not a rewrite.

Prompt to paste:

Create config/practice.yaml and config/appointment_types.yaml for Fairy Share Dental.

practice.yaml: name, Dallas TX, America/Chicago, office hours (Mon–Thu 8–5, Fri 8–2, lunch 12–1 blocked), booking rules (2h minimum notice, 21-day search limit, 30-minute start increments, offer 3 options, slot offers expire in 15 minutes), and the provider and operator IDs I found in Open Dental.

appointment_types.yaml: New Patient Exam 60m, Cleaning 60m, Emergency Exam 30m, General Consultation 30m, Cosmetic Consultation 45m. Each needs an Open Dental Pattern string (one character per 5 minutes), which provider role it needs, and which rooms it can use. Validate on load that the pattern length matches the duration, and fail loudly if not.

- Done when: the app refuses to start if a duration and its pattern disagree.

Step 4 — The Open Dental adapter

One file that translates to and from Open Dental, and makes no decisions of its own.

Prompt to paste:

Write the Open Dental adapter: backend/app/opendental/client.py plus a mock with the same interface.

Requirements:

- ODFHIR auth header from environment variables only
- Respect the Remote API throttle: serialize requests ~1.1s apart, retry 429s using Retry-After
- Page results with Offset (100 per page)
- Convert responses into small internal models and DROP every field we don't need — provider records include SSN, licence and NPI, and none of that may reach the agent or the logs
- The adapter translates only; all booking decisions live in the services layer

- Done when: a test proves a provider's SSN never appears in any adapter output.

- Film: this is a good 20-second beat about not leaking data you didn't ask for.

Step 5 — Availability, the honest way

Open Dental's slot finder needs provider schedules, which the test database does not have. So availability comes from the configured office hours, minus what is genuinely booked in Open Dental right now.

Prompt to paste:

```
Build the availability service. Two sources, switched by config:  
- slots: use Open Dental GET /appointments/Slots (for a practice whose schedules are  
  set up)  
- config: office hours from practice.yaml, MINUS live appointments fetched from Open  
  Dental
```

```
In both cases: generate candidate start times at the configured increment, drop  
anything before now + minimum notice, drop anything crossing lunch or closing, and  
exclude any candidate whose room OR whose provider is already busy. Return at most 3  
options, spread across different days, each with a spoken time string like "Tuesday,  
September 29th at 9 AM".
```

- Done when: booking a room at 8:00 makes the 8:00 option disappear for that room.
- Say on camera: "The office hours are mine. Every conflict check is real." That honesty is the point.

Step 6 — Check, book, verify

The heart of the build, and the heart of the video.

Prompt to paste:

```
Build the booking service with this exact contract:
```

```
create(): 1) if this conversation already booked this slot, return that booking instead  
of making a second one; 2) re-check the slot against live Open Dental (someone may have  
taken it since it was offered); 3) POST the appointment; 4) return status  
created_pending_verification. It must NEVER return anything that reads as "booked".
```

```
verify(): GET the appointment back from Open Dental and compare patient, time, room,  
provider, length and status against what we asked for. Then check no earlier  
appointment clashes with it – Open Dental does not document double-booking protection,  
so the lower appointment number wins. Only this function may return verified: true.
```

- Done when: tests cover the slot being taken mid-booking, the appointment vanishing, and a retry not creating two bookings.

- Film: delete the appointment behind the agent's back, then watch verify fail and the agent refuse to confirm.

Step 7 — The tools the agent is allowed to use

Prompt to paste:

Expose the six tools as POST endpoints under /tools, protected by a shared secret header.

Rules:

- The agent never receives raw Open Dental IDs. Hand it HMAC-signed opaque references instead (patient_ref, slot_id, appointment_ref), so it cannot invent or alter one. Slot offers expire after 15 minutes.
- Every response is HTTP 200 JSON with ok and agent_instruction, where agent_instruction says what the agent must and must not say next.
- Logs contain no names, dates of birth or phone numbers.
- lookup_patient returns only found / multiple / not_found – never record details.

agent_instruction is the trick worth explaining on camera: every tool answer carries its own instruction, so the safety rule travels with the data instead of living only in the prompt.

Step 8 — The voice prompt

Written for speech, not for reading. One question per turn, and no thinking out loud.

Prompt to paste:

Write the ElevenLabs system prompt for Joy at Fairy Share Dental.

Must include:

- Everything she outputs is SPOKEN. She must never say her reasoning, plans or tool names out loud.
- One question per turn. Short sentences. Natural dates: "Monday the 28th at 8".
- Booking flow: what they need → new or existing → name → date of birth → (existing: look them up) → preferences → check availability → offer two times → confirm the choice → book → verify → only then say they're booked → offer a written confirmation.
- THE CORE RULE: never say booked, scheduled or all set unless verify_appointment returned true in this call.
- Emergencies first: trouble breathing or swallowing, spreading swelling, uncontrolled bleeding or facial injury means tell them to call 911 and do not book.
- Privacy: discuss a record only after name and date of birth match. Never ask for SSNs or insurance IDs.

Pick the model carefully. Our first version used Gemini 2.5 Flash and it spoke its own notes aloud on the call: “The user is a new patient. I need to collect...”. Claude Haiku 4.5 fixed it. This is a real, filmable failure — see Part 5.

Step 9 — Create the agent from a script, not by clicking

Clicking through a dashboard is impossible to repeat on camera and impossible to redeploy. Script it.

Prompt to paste:

```
Write elevenlabs/provision_agent.py that uses the ElevenLabs API to create: the tool secret, the knowledge base document, all six webhook tools (20-second timeout, conversation_id bound to the system_conversation_id dynamic variable) and the agent itself with the system prompt.
```

```
Save the created IDs to a local file so re-runs resume instead of duplicating. Add --dry-run, --update (push a new prompt) and --retarget (point the tools at a new backend URL).
```

- Done when: one command produces a working agent, and --update pushes prompt edits in seconds.

Step 10 — Confirmations through n8n

Prompt to paste:

```
Add a send_confirmation tool and an n8n workflow.
```

```
Backend: re-verify the appointment in Open Dental FIRST; refuse to send anything for an unverified appointment. Validate the destination (email format, 10-digit US mobile). Never send the same confirmation twice. Treat the delivery as successful ONLY if the workflow explicitly replies sent: true.
```

```
n8n: a webhook protected by a header secret, a Code node that validates the payload and builds the message, a Switch that routes to Gmail / Twilio / Telegram, and a Respond node that reports back whether it was sent. The message contains first name, appointment type, time, provider and the practice number – never a date of birth or a patient ID.
```

No Twilio account? Add a demo setting that delivers “text me” confirmations to a Telegram chat instead. The agent still says it is texting; switching to real SMS later is one setting.

Step 11 — Deploy so the demo survives a reboot

A tunnel from your laptop is fine for the first test and painful after that: the URL changes on every restart, and every change means repointing the tools.

Prompt to paste:

```
Add a Dockerfile and a render.yaml blueprint for the backend: Docker runtime, free plan, health check at /health, all secrets marked sync: false so they live in the dashboard and never in the repo. Then tell me the exact steps to deploy it and what to paste where.
```

- Free instances sleep after ~15 minutes idle. Add an n8n schedule that pings /health every 10 minutes.
- Film: the deploy failing because the secrets were not set yet. It fails loudly instead of running unsafely.

Step 12 — Test the agent like a caller, not like a developer

Unit tests prove the backend. They say nothing about what the agent says out loud. ElevenLabs can run scripted callers against the live agent and grade the transcript against your own criteria.

Prompt to paste:

```
Write elevenlabs/simulate_calls.py using the ElevenLabs simulate-conversation API. Each scenario is a caller persona, a first message, optional mocked tool results, and pass/fail criteria graded from the transcript.
```

```
Scenarios: a medical emergency who pushes hard for an appointment; unreadable tool results; a verification that fails; a cracked tooth with no red flags; and a straightforward cleaning booking.
```

```
Note: the simulator MOCKS tools by default (every result is the string "Tool Called."), so supply realistic results via tool_mock_config, and cap the turns per scenario or it invents a second call after the caller hangs up.
```

This test found a real bug. Given an unreadable tool result, the agent invented two appointment times and told the caller they were booked. The rule now says: if you cannot read the times, you have no times; if you cannot read verified true, it is not booked. Re-tested, it refuses and offers a call back.

Step 13 — The proof page

Showing raw JSON proves it to developers. Showing a schedule proves it to everyone.

Prompt to paste:

Build a read-only /schedule page on the backend, password protected, that renders the live Open Dental schedule: time, length, appointment type, patient, provider, room, status, and whether the AI or a human booked it. Day/week/month views. It must never write anything.

- Film: split screen — the call on the left, this page refreshing on the right.

Part 3 — The rewritten video script

Same structure and timings as yours. Corrected where it did not match the build, with the real failure stories added. About 11–12 minutes.

0:00 — Cold open

Play the test call, about 25 seconds, ending on the appointment appearing in the schedule page. Label on screen: “Test call · fictional patient · test database”.

0:25 — Hook and intro

“That was a test call to an AI receptionist I built for a dental practice. It checked the schedule, booked a cleaning, and that appointment is now sitting in Open Dental — the software the front desk actually runs on.”

“I'm Ben, I build conversational AI systems. In this video I'll show you exactly how it's built: the voice agent, the tools it's allowed to use, the backend that talks to Open Dental, and the n8n workflow that sends the confirmation. Then I'll test it twice — once where it books, and once where it should refuse.”

1:00 — Why Open Dental and not Google Calendar

“A dental office runs its whole day inside a practice management system. Open Dental, Dentrix, Eaglesoft. The schedule, the patients, the providers all live there.”

“If the AI books into a separate calendar, someone still has to copy every booking across. The work just moves. So I set one rule: the AI reads and writes the same schedule the front desk uses.”

“I picked Open Dental because it publishes its API and hosts a free test database, so I could build against the real software without ever touching a real practice.”

2:00 — The system in one picture

Show the diagram: CALLER → JOY (ElevenLabs) → 6 TOOLS → BACKEND → OPEN DENTAL, with n8n hanging off the backend for confirmations.

“Four parts. Joy is the voice. The tools are the only actions she's allowed to take. The backend is the only thing that touches Open Dental — it checks, books and verifies. And n8n sends the confirmation afterwards.”

“Joy never touches the database. She can use the tools I gave her and nothing else. The model proposes; the backend decides.”

3:00 — Build 1: the agent and the prompt

“Her first line is: thank you for calling Fairy Share Dental, how can I help you today?”

“The prompt gives her a role, and three hard rules. One: everything she outputs is spoken, so she never narrates her own thinking. Two: one question at a time, because stacked questions feel robotic on a phone call. Three: emergencies come first — trouble breathing, spreading swelling, uncontrolled bleeding means call 911, and don't book.”

“And a knowledge base: hours, location, services. When someone asks if we're open Saturday, she answers from that document instead of guessing.”

5:00 — Build 2: connecting Open Dental

Screen: the Open Dental API docs, then your backend code, then the n8n workflow. Blur every key.

“Open Dental's API uses a developer key and a customer key. I built against their test database — never build against a live office.”

“Find the patient: name and date of birth. No match, and she treats them as new and collects details.”

“Check availability: office hours, minus everything genuinely booked in Open Dental, and it returns three times, not twenty. Give a voice agent twenty options and it'll read all twenty out loud.”

“Book: the backend re-checks the slot, then creates the appointment. And here's the important part — creating it does not mean it's booked. The tool literally cannot say booked. It says pending verification.”

“Verify: read the appointment back out of Open Dental and compare the patient, the time, the room, the provider, the length. Only if all of that matches does Joy get to say you're booked.”

6:30 — What broke

Pick two. The first is the funniest, the third is the most serious.

- “The schedule was empty.” Open Dental's slot finder returned nothing, every time. The test database has no provider schedules, and you can't create them through the API. So availability comes from configured office hours, minus real appointments — every conflict check is still real.
- “She read her own notes out loud.” The first model narrated its thinking on the call: “The user is a new patient. I need to collect...”. Switching models fixed it, and the prompt now says everything she outputs is spoken.
- “It nearly lied about a text message.” The confirmation workflow could return a normal OK with an empty body when it had actually failed, which the backend counted as sent. Now only an explicit sent: true counts. That's the same bug as claiming an appointment is booked — a success that was never checked.

“If this is useful, subscribe — I'm building the Dentrix version next.”

8:00 — The live test

One uncut call, schedule page open beside it. Call as an existing test patient and ask for a cleaning. Pause on the appointment appearing.

“Same patient, same time, in Open Dental. And notice she asked if I wanted a text confirmation — that goes out only after the appointment is verified.”

Then the call that matters: “I have facial swelling and I'm finding it hard to breathe.”

“She gave the emergency instruction, didn't try to diagnose me, and didn't book a routine appointment. That's the behaviour I built and tested for.”

Test this call privately before you film it. If the model wanders, tighten the emergency section of the prompt and push it again before recording.

10:00 — Before a real practice uses this

Keep your checklist as written; it is accurate. Read it as four ticks:

- A HIPAA-eligible setup from your voice provider, with a signed BAA.
- API keys stored as secrets, never written into the prompt.
- A human fallback, so any caller can reach the front desk.
- Logging, so the office can see every booking the AI made.

“Everything here used a test database and invented patients. I'm not giving legal advice — work through this with the practice's compliance person.”

11:00 — Close

“So that's the build: a voice agent, a small set of tools that limit what she can do, a backend that verifies before anyone hears the word booked, and a confirmation that only goes out once the appointment is real.”

“The same pattern works with Dentrix or Eaglesoft, or any system with an API. Swap the tools, keep the rules.”

Part 4 — Prompt library

The prompts from Part 2, in order, ready to paste into Claude Code. Give it one step at a time and review what comes back; that review is the part worth showing on camera.

Step	Prompt purpose
1	Research the real API first; stop before coding
2	Verify the test environment by hand
3	Practice config: hours, rooms, providers, appointment types
4	Open Dental adapter with throttling and field stripping
5	Availability from hours minus live appointments
6	Check → book → verify booking service
7	The six tool endpoints with signed references
8	The spoken system prompt
9	Script the agent creation
10	Confirmations through n8n
11	Docker + deployment
12	Scripted call tests against the live agent
13	The read-only schedule page

Two prompts worth reusing on any build

Prompt to paste:

```
Before writing code, research the CURRENT official documentation for <service>. Tell me which of my requirements it can actually support, which endpoints we need, and anything you cannot verify. Do not invent endpoints or parameters. Then STOP and wait for my approval.
```

Prompt to paste:

Write tests for the failure paths before we go further: what happens when the slot is taken between offering and booking, when the record vanishes after we create it, when the API is down, and when the same request is retried twice. I want it to fail safely, not optimistically.

Part 5 — Demo day checklist

Run through this in the hour before you record.

- Re-seed the demo patients. The Open Dental test database is wiped nightly, so yesterday's patients and appointments are gone.
- Open /health and check it says live, config and confirmations enabled.
- Check the backend is awake — free hosting sleeps, and a cold start takes about a minute.
- Make one throwaway booking to warm everything up, then look at it on the schedule page.
- Test the emergency call once, privately.
- Regenerate the Meta WhatsApp token and paste it into the n8n credential. It expires every 24 hours.
- Message the WhatsApp test number from your phone. Meta only allows free-form messages within 24 hours of the person writing in — outside that window the API accepts the message and it never arrives.
- Check your WhatsApp is receiving confirmations.
- Have the schedule page open on a second screen, already logged in.
- Blur or crop: API keys, the .env file, the Render dashboard, your Telegram chat ID.

Recording notes

- Book a time of day that is genuinely free, or the agent will correctly refuse and you will have to re-record.
- Speak your test patient's date of birth clearly; voice models mishear years more than names.
- Let the pauses breathe. The two-to-five-second wait while it checks the schedule is honest, and cutting it makes the demo look faked.
- Say “test database” and “fictional patient” at least twice. It protects you and it builds trust.

Part 6 — If someone asks for it for a real practice

- Their Open Dental needs the eConnector running, a customer key enabled, and a paid tier for write access (per location, paid by the practice).
- You need a developer key from Open Dental vendor relations, and they require a signed BAA.
- Switch availability to Open Dental's slot finder once their provider schedules are set up.
- Add the parts V1 deliberately left out: reschedule, cancel, human transfer, and urgent-call escalation.
- Confirmations move from Telegram to real SMS and email, which means a Twilio account and US carrier registration.
- Never demo with real patient data, not even once.